capital

Custom Report Development

Software Version 2015.1

February 2016

© 2016 **Mentor Graphics Corporation**
All rights reserved.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS CORPORATION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

RESTRICTED RIGHTS LEGEND 03/97

U.S. Government Restricted Rights. The SOFTWARE and documentation have been developed entirely at private expense and are commercial computer software provided with restricted rights. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement provided with the software pursuant to DFARS 227.7202- 3(a) or as set forth in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable.

Contractor/manufacturer is:
Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777.
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: www.mentor.com/supportnet
Contact Your Technical Writer: www.mentor.com/supportnet/documentation/reply_form.cfm

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other third parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the respective third-party owner. The use herein of a third- party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at:
www.mentor.com/terms_conditions/trademarks.cfm.

End-User License Agreement: You can print a copy of the End-User License Agreement from:
www.mentor.com/terms_conditions/enduser.cfm.

Table Of Contents

Introduction 2

Interfaces 3

- **IXQuery** 3
- **IXQueryFactory** 5
- **IXReport** 6
- **IXResultExpression** 8
- **IXReportTemplate** 9
- **IXReportModule** 9
- **IXQueryExpressionFactory** 11
- **IXReportFactory** 13
- **IXReportProcessor** 13
- **IXReportProcessorPlugin.** 14
- **IXUserSessionMgr** 15
- **IXReportCreator** 16
- **IXEmbeddableReport** 17
- **IXLogicReport** 17
- **IXTopologyReport** 17
- **IXIntegratorReport** 18
- **IXHarnessReport** 18
- **IXSBOMReport** 18
- **IXHarnessTVMReport** 18
- **IXDataExchange** 19
- **IXDataExchangeRegistrarResult** 19
- **IXDataExchangeRegistrar** 19
- **IXPLMXMLTransformer** 20
- **IXTeamcenterReportExchangeRegistrarResult** 20
- **IXTeamcenterReportExchangeRegistrar** 21

JSP tags 21

- **Output XML Representation** 25
- **Deployment of a report JSP**: 27

Guidance 28

- **Here are some recommendations that will help you** 28

Examples 29

- **Grouping and Sorting using Custom Tags**: 31
- **Custom Attribute** 33
- **Custom Embedded Report Plugin Development** 38

Appendix A 40

Introduction

This document describes how you can develop custom reports for Capital Enterprise Reporter by using extensible interfaces for querying and reporting. Capital Enterprise Reporter is a web-based reporting app server that allows users to create and deploy servlets for querying and reporting. It uses Jetty webserver to host web applications which can access Capital data using extensibility interfaces. Web applications are provided with a servlet context that allows them to connect to Capital Manager, form queries and fetch Capital data in the form of extensibility objects. Furthermore, they have access to extensibility interfaces and JSP tags which work on the extensibility objects to define reports. We divide this document into two parts the first of which describes extensibility classes for querying and the second for reporting.

Interfaces

IXQuery

IXQuery interface is used to fetch extensibility objects of a particular type by providing its name, its extensibility class and a filter condition (IXFilterExpression). In addition, queries may work on a scope that helps isolate the data to be fetched by providing a “related query”. For example one can provide a query for particular project as a related query to a query for designs, which when executed results in designs of that particular project. Query for projects will not require a scope (i.e. no related queries need be provided). All others require at least one scoping query. One may add as many related queries as needed. Executing a query will multiple related queries will result in a union of results from each related query. An internal relational framework looks up how entities resulting from one query are related to another, and this is most often predefined. The set of such relations appears in the appendix below. For example wires are related to connectors through their connections from their pins, so querying for wires with a related query for connectors will fetch wires attached to the connectors only.

```
public interface IXQuery
{

        String getEntityName();

        Class<? extends IXObject> getEntityClass();

        IXFilterExpression getFilterExpression();

        Collection<IXQuery> getRelatedQueries();

        Collection<IXObject> getResult();

        void addRelatedQuery(IXQuery relatedQuery);

        void removeRelatedQuery(IXQuery query);

}
```

getEntityName() – get the name of the entity ("Wire/Net/Device" etc) that defines this query. This name would match the html name specified in doc\plugin\api\Attributelist.html

getEntityClass() – get extensibility class of the entity to query on.

getRelatedQueries() -- get all related queries that define the scope for the data to returned by this query. A related query is expected to return objects that are related to entity this query is expected to fetch. This helps isolate objects that are returned by this query to those that are related to objects returned by the related query.

getFilterExpression() – get the filter expression defining this query. A filter expression is used to narrow down data returned by this query to those that satisfy the condition specified in the expression. Custom implementations of IXFilterExpression can also be provided while forming queries.

getResult() – Execute the query to return a list of entities which satisfy the filter expression for the defined scope.

addRelatedQuery(IXQuery relatedQuery) – Append a query to the list of related queries that scope the data returned by this query. A related query is expected to return objects that are related to entity this query is expected to fetch. A related query is used to narrow down the objects that are returned by this query to those that are related to objects returned by the related query, Relations are defined by the extensibility methods of these objects. For ownership and containment relations it is possible to skip intermediate queries. For example, one can query Cavity Detail information directly from a Harness Design by adding a related query for Harness Design for Cavity Detail query without having to look for them in wire ends, or cavities of connectors, Similarly, one can query for a multicore directly from a connector without having to add intermediate queries for connecting pins, wires.

IXFilterExpression

A filter expression is used by a query (see IXQuery) to filter down data fetched by queries to those that satisfy the condition specified in the expression. System filter expressions (boolean returning query expressions available under system node in Capital Project) are available to server pages through the interface IXQueryExpressionFactory.

```
public interface IXFilterExpression {

        boolean isSatisfiedBy(IXObject entity);

        String getName();

        String getCollapsedForm();

}
```

isSatisfiedBy(IXObject entity) – Evaluate the filter expression for the supplied entity. It returns true if the entity satisfies the filter condition, otherwise returns false.

getName() – Get the name for this expression.(may be empty). For a system expression this the same as seen in query expression editor.

getCollapsedForm() – get the collapsed form of this expression as a string. For system expression the collapsed form is the same as seen in query expression editor in the collapsed view. For example a filter expression that checks if the attribute Harness equals H1, would have a collapsed form

(attribute 'Harness') = 'H1'.

IXQueryFactory

As the name suggests an IXQueryFactory interface is used to create queries. An instance of query factory is available to server pages as an attribute in the HTTP session after the user is logged in. (for connecting to manager and initialization, see IXUserSessionMgr:connect).

```
public interface IXQueryFactory
{
        IXQuery createQuery(Class<? extends IXObject> xClass, String entityHTMLName, String
filterExpName)      throws NoSuchFieldException;

        IXQuery createQuery(Class<? extends IXObject> xClass, String entityHTMLName,
IXFilterExpression filterExpression)      throws NoSuchFieldException;
}
```

`createQuery(Class<? extends IXObject> xClass, String entityHTMLName, String filterExpName)` – This method creates a query using a extensibility class, extensibility entity name and a system filter expression (boolean query expressions in system data in Capital Project). If the entity name or class is not found a `NoSuchFieldException` is thrown. The argument 'filterExpName' may be empty or null, in which case a query is created without any filer condition.

`createQuery(Class<? extends IXObject> xClass, String entityHTMLName, IXFilterExpression filterExpression)` – This method creates a query using a extensibility class, extensibility entity name and a user implemented filter expression. If the entity name or class is not found a `NoSuchFieldException` is thrown. The argument 'filterExpName' may be empty or null, in which case a query is created without any filer condition.

The list of entity names that can be queried are listed in appendix A:

Following code shows how to access the query factory instance in a java server page.

```
IXQueryFactory queryFactory = (IXQueryFactory)session.getAttribute("queryFactory");
```

Here `session` is the HTTPSession object available in JSP pages.

IXReport

An `IXReport` object can be used to produce xml content in a format ready for styling using a stylesheet. It defines report content and its output xml structure independent of how or where the data to report is got from. It is only at the time of evaluation of a report that the "where from" and "how" information is provided in the form of queries (IXQuery collection) and a report processor (IXReportProcessor). Typically, a report would consist of a set of tables with columns and rows. In the same spirit, an `IXReport` consists of a set of `IXReportModule` objects with column data defined using `IXReportTemplate` objects. Likewise the output xml content that it produces on evaluation, also has top level report element with a set of modules, report headers and result-sets as rows filled in for each entity to report on.

To report modules we entrust the task of defining the types of the entities which we wish to report. This is done by adding queries to the report module for each of the types of entities we wish to show in the report along with any filter conditions. For example in a Tube BOM report we would like to show insulations of the type tube, so we specify a query for entities of type insulation with a filter condition that picks only those of the type tube. The report module additionally defines column content using `IXReportTemplate` objects.
When a report is evaluated the report module, the report processor decides how entities in the scoping query are related to entities to be shown in a report. For example, for a wire list report if the scoping query is for connectors, the report processor "Directly Connected" would fetch wires attached to pins of the connectors returned by the scoping query. For each of these wires, column expressions (`IXReportTemplate` objects) are evaluated to return rows as xml strings.

It is possible to add custom tags and attributes at any level (to `IXReport`/`IXReportModule` or `IXReportTemplate`) in report definition. These attributes are written out at the same level in the output xml on report evaluation. For example, you may want to add a tag in a report module

specifying which column you would like to group by. This tag or the attribute is written in the output xml in the report module.

```
public interface IXReport {

    void addModule(IXReportModule module);

    String evaluate(Collection<IXQuery> queries, IXReportProcessor processor);

    List<IXReportModule> getModules();

    String getTitle();

    void addCustomAttribute( String name, String  value);

    String getCustomAttributeValue( String attributeName);

    Set<String> getCustomAttributes();

    void addCustomtag(String tag, Map<String,String> attributes);

    Set<String>  getCustomtags();

    List<Map<String,String> >  getAttributesForCustomtag(String tag);
}
```

addModule(IXReportModule module) – add a new module to the end of the list. (see IXReportModule).

evaluate(Collection<IXQuery> queries, IXReportProcessor processor) - evaluate the report to produce XML string output. Evaluation proceeds by first evaluating queries supplied as scope. For each report module, the report processor is used to return the entities queried in the module. It then applies result expressions in report templates on each of these entities to populate the rows of the report.

getModules() - get the reporting modules contained in this report. (see IXReportModule)

getTitle() - get the title of this report.

addCustomAttribute(String name, String value) - add a custom attribute name and value to the report. Custom attributes provides clues to formatter to style xml content resulting from report evaluation.

getCustomAttributeValue(String attributeName) – get a custom attribute value (string) for the given attribute name. If the attributeName is not found, this function returns null.

getCustomAttributes() – get a list of custom attributes value for the given attribute name

addCustomtag(String tag, Map<String,String> attributes) – add a user defined tag on the module. A tag is useful typically as a hint to xml formatter after xml report is generated. For example a custom tag 'groupBy' with attribute title='XXX' might indicate which column to group by while formatting. Note: tag names need not be distinct.

getCustomtags() – get all user defined tags in the report module definition(added using the method addCustomtag)

getAttributesForCustomtag(String tagName) – get the list of name value pairs associated with a custom tag name. Since tags may be duplicated, there may be more than one entry in list returned.

IXResultExpression

Result expressions evaluate on extensibility objects to return strings or collection of strings. It is used for example in report templates(IXReportTemplate) as expressions that define column content in a table.

evaluate(IXObject entity) – evaluate the expression for the supplied entity. It may either return a string or a collection of strings.

evaluate(IXObject entity, String executionContext) – This is a variant of the evaluate function above that takes in a hint based on which it may choose to evaluate differently. As for the other evaluate function it may either return a string or a collection of strings.

getName() – An optional name for this result expression(may be empty)

IXReportTemplate

A report template has a result expression(IXResultExpression) which is a formula that decides the content(string) to be shown for the entity
types queried for in a report module. In a tabular report, the report template decides the content that goes into a single column of the table. In the evaluation of a report, the result expressions defining the columns are applied on each entity resulting from queries of a module, returning strings that form the rows of the table.

```
public interface IXReportTemplate {

    IXResultExpression getResultExpression();

    String getTitle();

    Set<String> getCustomAttributes();

    void addCustomAttribute( String name, String  value);

    String getCustomAttributeValue( String attributeName);
}
```

addCustomAttribute(String name, String value) - add a custom attribute name and value to the report. Custom attributes provides clues to formatter to style xml content resulting from report evaluation.

getCustomAttributeValue(String attributeName) – get a custom attribute value (string) for the given attribute name. If the attributeName is not found, this function returns null.

getCustomAttributes() – get a list of custom attributes value for the given attribute name

IXReportModule

A reportmodule groups report content based on entity types to report on which a set of formulas called result expressions which apply on these entity types. More precisely, it has a collection of queries (which may have filters) that define what entity types are to be shown in a report module and a set of IXReportTemplate objects which are thin wrappers around result expressions(IXResultExpression) that apply on the entities returned by queries. There can be more than one report module inside a report. A report module typically corresponds to a single table in the real world, the columns of which are defined by the expressions in IXReportTemplate objects. So when a report is evaluated, the report module fetches entities using the queries defined in the module (see IXReportfor further details on report evaluation) and applies the expressions in report templates for each resulting entities for form row sets. As with real world tables, they can be nested, so one can have a module inside a module. Nested modules queries are scoped not only by the external scoping queries (obtained as an argument to evaluate method in IXReport) but also by the queries in the parent module. Finally a report module may have a title and custom tags and attributes.

```
public interface IXReportModule {

    Collection<IXQuery> getQueries();

    void addQuery( IXQuery query , String queryTag);

    Set<String> getQueryTags();

    Collection<IXQuery> getQueriesForTag(String queryTag);

    Collection<IXReportTemplate> getReportTemplates();

    String getTitle();

    void addCustomAttribute( String name, String  value);

    String getCustomAttributeValue( String attributeName);

    Set<String> getCustomAttributes();

    void addCustomtag(String tag, Map<String,String> attributes);

    Set<String>  getCustomtags();

    List<Map<String,String> >  getAttributesForCustomtag(String tagName);

    void addReportTemplate(IXReportTemplate reportTemplate);

    void addReportModule(IXReportModule reportModule);

    Collection<? extends IXReportModule> getReportModules();
}
```

getQueries() – get a collection of queries for entity types to show in this report module. For example a wire list report would have a module with queries for wires and shields.

addQuery(IXQuery query , String queryTag) – add a query to report module with an optional tag to query. A report module can have duplicate queries for a particular entity type. To differentiate the results from duplicate queries, queries can be tagged in a module. These tags are optional and may be empty ("). On evaluation of a report these tags as used as hints for result expressions (see IXResultExpression:evaluate(IXObject entity, String executionContext) to evaluate differently on objects from queries for the same entity type. For example in component BOM report, one can query for IXCavityDetail twice in a module, and then while evaluating the result expression for displaying part number the query tag is passed as a hint to show SealPartNumber for the IXCavityDetail got from the first query and TerminalPartNumber for the IXCavityDetail resulting from the second query.

getQueryTags() – get a collection of tags on query return types. see addQuery for description on tags.

getQueriesForTag(String queryTag) – get the collection queries that have the specified tag. See addQuery for description on tags

getReportTemplates() – get the ordered list of report templates defined for this report. The same order for the columns in the report will be maintained in the report generated.

getTitle() – get the title of this report.

addCustomAttribute(String name, String value) – Add a custom attribute name, value (as strings) for this report template.

getCustomAttributeValue(String attributeName) – Get custom attribute value for the given attribute name. This returns null if the attributeName provided is not found.

getCustomAttributes() – Get user defined attribute names for the module as a collection.

addCustomtag(String tag, Map<String,String> attributes) – add a user defined tag on the module. A tag is useful typically as a hint to xml formatter after xml report is generated. For example a custom tag 'groupBy' with attribute title='XXX' might indicate which column to group by while formatting. Note: tag names need not be distinct.

getCustomtags() – get all user defined tags in the report module definition(added using the method addCustomtag)

getAttributesForCustomtag(String tagName) – get the list of attributes as name -value pairs associated with a custom tag name. Since tags may be duplicated, there may be more than one entry in list returned.

addReportTemplate(IXReportTemplate reportTemplate) – Append a report template as the last report template of the module. A report template typically defines column content using a result expression.

addReportModule(IXReportModule reportModule) – Modules could be nested by adding one within the other. Use addReportModule to add a report module (sub-module) within this module. In the evaluation of this report module, each object returned by queries associated with this module goes in as scope for queries defined in sub-module. So for example if this report module is a wire list that returns wires and a sub-module queries on connectors, wires become the scope for the sub-module query that returns connectors and so connectors related to wires are returned in evaluating the sub-module query.

getReportModules() – get a collection of all nested modules in this report module.

IXQueryExpressionFactory

IXQueryExpressionMgr interface can be used to access system defined expressions and also to create expressions. These expressions can then be used to provide filter conditions to queries (IXFilterExpression) or as expressions that define column content in reports (IXResultExpression).

```
public interface IXQueryExpressionFactory
{

    Collection<IXFilterExpression> getSystemFilterExpressions(Class<? extends IXObject> entityClass);

    Collection<IXResultExpression> getSystemResultExpressions();

    IXFilterExpression parseFilterExpression(String expression) throws ParseException;

    IXResultExpression parseResultExpression(String expression) throws ParseException;

}
```

getSystemResultExpressions() - get a collection of system defined result expressions(implementing IXResultExpression) .System defined query expressions are non-boolean returning query expressions that appear under the system data node in Capital Project.

getSystemFilterExpressions(Class<? extends IXObject> entityClass) – get a collection of pre-defined filter expressions. Pre-defined filter expressions are boolean returning query expressions that appear under the system data node in Capital Project. entityClass specifies the type of entity on which the expressions apply. When entityClass is null this function returns all system defined boolean returning expressions available.

parseFilterExpression(String expression) – parse the xml string form of a filter expression (as defined in project.dtd for queryexpression and its operators) and return a filter expression(IXFilterExpression). It throws a ParseException if the string does not evaluate to a valid expression. For example

```
<queryExpression>
  <equals>
        <attributeVariable name="Harness"/>
       <stringConstant value="H1"/>
  </equals>
</queryExpression>
```

is the xml representation of a filter expression that returns true on a XObject if its attribute is equal to H1.

parseResultExpression(String expression) – parse the xml string form of a filter expression (as defined in project.dtd for queryexpression and its operators) and return a filter expression(IXFilterExpression). It throws a ParseException if the string does not evaluate to a valid expression. For example

```
<queryExpression> <attributeVariable name="Harness"/>
</queryExpression>
```

The above example is the xml representation of a result expression that returns the value of the attribute "Harness" of an extensibility object.

IXReportFactory

The report factory instance can used to create reports (IXReport) and its contents (IXReportModule and IXReportTemplate). An instance of a report factory is available to all server pages from the session after a user is logged in.

```
public interface IXReportFactory
{
    IXReport createReport(String title) ;

    IXReportModule createReportModule(String title);

    IXReportTemplate createReportTemplate(IXResultExpression expression, String title) ;

    IXReportTemplate createReportTemplate(String expressionName, String title) throws
NoSuchFieldException;

}
```

createReport(String title) – create an empty report (IXReport) with a title.

createReportModule(String title) – create an empty report module (IXReportModule) with a title.

createReportTemplate(IXResultExpression expression, String title) – Create report template with an expression and a title. Expressions in report templates are evaluated for each object returned by the queries defined for the report, thus populating typically a column of a table (see IXReportTemplate)

createReportTemplate(String expressionName, String title) throws NoSuchFieldException – Create report template with title, a query expression name that refers to query expression from system data(system query expressions are found under system data node in Capital Project). Expressions in report templates are evaluated for each object returned by the queries defined for the report, thus populating typically a column of a table (see IXReportTemplate). A NoSuchFieldException is thrown when the expression name (expressionName) provided as argument is not found in system data.

Following code shows how to access the query factory instance in a java server page.

```
IXReportFactory reportFactory =      (IXReportFactory)session.getAttribute("reportFactory");
```

IXReportProcessor

Report processors decide how entities queried for in reports are related to those returned by the scoping queries.
A custom processor for report generation can implement this interface to define how query results

from scoping queries are related to entities queried for in reports. Standard processors namely, "Directly Connected", "Connectivity between Pin lists", and "No Processor" implement this interface and are made available to server pages as attributes in HTTP session. The processor "Connectivity between Pin lists" processor also honors plug-ins that implement the interface IXReportProcessorPlugin.

```
public interface IXReportProcessor
{

    String getName();

    void setUsePlugin(String pluginName);

    Collection<? extends IXObject> getRelatedObjects(
                        String targetEntityName,
                        Class<? extends IXObject> targetEntityClass,
                        Collection<? extends IXObject> inputSet,
                        String context);
}
```

getName() – get the name of this processor.

setUsePlugin(String pluginName) – customize the behavior of this report processor with a plug-in reference.(see IXReportProcessorPlugin).

getRelatedObjects(
String targetEntityName,
Class<? extends IXObject> targetEntityClass,
Collection<? extends IXObject> inputSet,
String context) – get a collection of objects of the type specified by targetEntityName and targetEntityClass that are related to objects in inputSet in a sense determined by the processor. Each query in a report module that queries for objects of type specified by targetEntityName and targetEntityClass would use this method to fetch objects of the targetEntityType. The additional argument context is provided as hint to evaluate based on context. In report evaluation, this context is the same as the tag attached to each query in report module. For example, in component BOM report for a logic design, we would not want shared objects to appear more than once even though multiple extensibility objects for wires might be got from different designs. To fix this, queries in component BOM are given tags "discardSharedDuplicates" that report processors can recognize and discard shared duplicates before returning these objects.

Following code shows how to get standard report processors from a HTTP session

IXReportProcessorPlugin.

As the standard processor "Connectivity between Pinlists" discovers connecting paths (and

entities on the path) available from one pinlist to other, it usually jumps over in-lines and splices to navigate further to see if there is connecting path. Plugins can customize this by instructing the report processor that it can continue or stop when it encounters pinlists like fuses or relays. "Connectivity between pinlists" processor can continue processing by jumping over pins, if a connected pin (or pins) is provided. A plug-in can provide a list of connected pins when the processor encounters a pin. If empty the processing will stop at this pin.

```
public interface IXReportProcessorPlugin extends IXPlugin
{

    List<IXAbstractPin> getConnectedPin(IXAbstractPin pin);

}
```

getConnectedPin(IXAbstractPin pin) get connected pins for the given pin, from where to continue processing whilst navigating connectivity.

IXUserSessionMgr

The interface IXUserSessionMgr provides methods to connect to and disconnect from CapitalManager.

```
public interface IXUserSessionMgr {

String connect( String username, String password);
void disconnect(String sessionID);

}
```

connect(String username, String password) throws Exception – Connect to Capital Manager with a Capital user name and a password. One successful login, a user session id is returned as a string. An exception is thrown if the connection to manager fails.

disconnect(String sessionID) – Disconnect a user session from Capital Manager by providing its dither session id is the same the one got when a connection is established. Web applications may call this method when user logs out or session destroyed either by timeout or as user closes web browser.

The singleton instance of a user session mgr is available as an attribute in the servlet context can be accessed as below.

```
IXUserSessionMgr userSessionMgr = session.getServletContext().getAttribute("userSessionMgr");

String sessionID= userSessionMgr.connect(username,password);
```

After login, to obtain the factories for creating queries/reports, set the session ID as a session attribute. The factories can then be accessed as described earlier. Following code shows how to set user session id in the session

```
session.setAttribute("userSessionId",sessionID);
```

IXReportContext

In order to define a custom embedded report, this is the interface that will provide factory classes to define report.

```
public interface IXReportContext{

        IXReportFactory getReportFactory();

        IXQueryFactory getQueryFactory();

        IXQueryExpressionFactory getQueryExpressionFactory();

        IXLibrary getLibrary();
}
```

getReportFactory(): returns report factory instance, which can used to create reports (IXReport) and its contents (IXReportModule and IXReportTemplate). For more details see IXReportFactory

getQueryFactory(): returns query factory instance, this is used to create queries. For more details see IXQueryFactory

getQueryExpressionFactory(): returns query expression factory instance, this used to access system defined expressions and also to create expressions. These expressions can then be used to provide filter conditions to queries. For more details see IXQueryExpressionFactory

getLibrary(): returns library factory instance, this is used to access the library data.

IXReportCreator

In order to build a custom report plugin, you will need to implement IXReportCreator.

```
public interface IXReportCreator extends IXPlugin {

        IXReport createReport (IXReportContext context);
}
```

IXReportCreator extends from IXPlugin interface. Hence getName(), getVersion() and getDescription() methods need to be implemented. Additionally, following method need to be implemented:

createReport (): will define how report will look like. An IXReport object can be used to produce xml report content. It defines report content and its output xml structure independent of how or

where the data to report is got from. It is only at the time of evaluation of a report that the "where from" and "how" information is provided in the form of queries (IXQuery collection). Typically, a report would consist of a set of tables with columns and rows. In the same spirit, an IXReport consists of a set of IXReportModule objects with column data defined using IXReportTemplate objects. Likewise the output xml content that it produces on evaluation, also has top level report element with a set of modules, report headers and result-sets as rows filled in for each entity to report on. (See IXReport for more details)

IXEmbeddableReport

In order to build a custom embedded report plugin, you will need to implement one of the interfaces that extend IXEmbeddableReport.

```
public interface IXEmbeddableReport extends IXReportCreator {

        StreamSource getStyleSheet ();
}
```

As you can see IXEmbeddableReport extends the IXReportCreator interface. This means that you will need to implement the getName(), getVersion(), getDescription() and createReport() methods. Additionally, you will need to implement the following method:

getStyleSheet ():will be called at the end of the report generation operation to convert report xml to html format.

You should not implement IXEmbeddableReport directly; you need to implement one of the following interfaces IXLogicReport, IXHarnessReport, IXIntegratorReport, IXTopologyReport. These interfaces do not add any additional methods. However, they do indicate the type of design your custom report is meant to generate.

IXLogicReport

To implement a custom embedded report for Logic designs or build lists, you will need to implement the IXLogicReport. By implementing IXLogicReport, the custom plugin will be automatically associated with embedded reports for logic designs.

```
public interface IXLogicReport  extends IXEmbeddableReport{

}
```

IXTopologyReport

To implement a custom embedded report for topology design, you will need to implement the IXTopologyReport. By implementing IXTopologyReport, the custom report plugin will be automatically associated with embedded reports for topology designs.

```
public interface IXTopologyReport extends IXEmbeddableReport {

}
```

IXIntegratorReport

To implement a custom embedded report for integrator design, you will need to implement the IXIntegratorReport. By implementing IXIntegratorReport, the custom report plugin will be automatically associated with embedded reports for integrator designs.

```
public interface IXIntegratorReport extends IXEmbeddableReport {

}
```

IXHarnessReport

To implement a custom embedded report for Harness designs or build lists, you will need to implement the IXHarnessReport. By implementing IXHarnessReport, the custom plugin will be automatically associated with embedded reports for harness designs.

```
public interface IXHarnessReport extends IXEmbeddableReport{

}
```

IXSBOMReport

To implement a custom embedded report for an SBOM, you will need to implement the IXSBOMReport. By implementing IXSBOMReport, the custom plugin will be automatically associated with embedded reports for SBOMs.

```
public interface IXSBOMReport extends IXEmbeddableReport{

}
```

IXHarnessTVMReport

To implement a custom embedded report for HarnessTVM cost results, you will need to implement the IXHarnessTVMReport. By implementing IXHarnessTVMReport, the custom plugin will be automatically associated with embedded reports for HarnessTVM cost results.

```
public interface IXSHarnessTVMReport extends IXEmbeddableReport{

}
```

There is nothing stopping you from creating a single custom embedded report plugin that can be used in multiple applications. For example, by implementing both the IXLogicReport and IXHarnessReport interfaces, the custom report plugin you develop will be associated with Logic and Harness designs.

IXDataExchange

You need to implement this interface if the intent is to exchange the data from Capital with external systems like PDM tools, disk save, etc. You will need to register the IXDataExchange with the help of IXDataExhangeRegistrar.

```
public interface IXDataExchange extends IXPlugin
{
    void bridgeOut(InputStream dataTobeExchanged);
}
```

As IXDataExchange extends from IXPlugin, you need to implement getName (), getVersion () & getDescription (). Additionally you will have to implement the following method:

bridgeOut (InputStream dataTobeExchanged)

@dataTobeExchanged – The data to be exchanged. For example in case of custom report exchange, the data will be report xml content populated by the Report Creator.

IXDataExchangeRegistrarResult

This interface should not be implemented. The IXDataExchangeRegistrarResult object will be created by Capital and passed onto the Registrar plugins to register the results.

```
@NotToBeImplemented
public interface IXDataExchangeRegistrarResult
{
    void add(String name, Class<? extends IXDataExchange> exchanger,
                        Class<? extends IXReportCreator> reportCreator);
}
```

add (…): This method will be called by the Registrar implementer to add new Report Creator and Data Exchange.

@name: This name will be used by Capital to populate new actions menu.
@exchanger: The IXDataExchange plugin implementation class
@reportCreator: The IXReportCreator plugin implementation class

IXDataExchangeRegistrar

You need to implement this interface to register the Report Creator and the Data Exchange. The implementer can register multiple reports with the same data exchange or multiple data exchange and vice-versa. The name of this plugin implementation will be used to populate a menu in Capital tools. Under the menus all the registered report names will be shown as separate action.

```
public interface IXDataExchangeRegistrar extends IXPlugin
{
    void register(IXDataExchangeRegistrarResult result);
}
```

register (IXDataExchangeRegistrarResult result): This method will be called to register all the data exchange registrar results.

@ result: The result object to be populated by the implementer

Following code shows how to register the result:

```
result.add ("WireList CSV Export", ReportToCSVFileExchange.class, SimpleWireListReport.class);

        ReportToCSVFileExchange is the class implementing the IXDataExchange plugin.
        SimpleWireListReport is the class implementing the IXReportCreator plugin.
```

IXPLMXMLTransformer

You need to implement this interface if the intent is to exchange the data (Report) from Capital to Teamcenter. You will need to register the IXPLMXMLTransformer with the help of IXTeamcneterReportExchangeRegistrar.

```
public interface IXPLMXMLTransformer extends IXPlugin
{
    InputStream transformToPLMXML(InputStream dataToBeExchanged);
}
```

InputStream transformToPLMXML(InputStream dataToBeExchanged):

@dataTobeExchanged – The data to be exchanged. For example in case of custom report exchange, the data will be report xml content populated by the Report Creator.

@return : It is up to the implementer to convert the report output to the appropriate PLMXML format and return it

IXTeamcenterReportExchangeRegistrarResult

This interface should not be implemented. This is created by Capital and passed onto the Registrar plugins to register the results.

```
@NotToBeImplemented
public interface IXTeamcenterReportExchangeRegistrarResult
{
    void add(String reportName, Class<? extends IXPLMXMLTransformer> transformer,
                        Class<? extends IXReportCreator> reportCreator);
}
```

add (...): This method will be called the Registrar implementer to add new Report Creator and Data Exchange.

@name: This name will be used by Capital to populate new actions menu.

@transformer: The IXPLMXMLTransformerplugin implementation class

@reportCreator: The IXReportCreator plugin implementation class

IXTeamcenterReportExchangeRegistrar

You need to implement this interface to register the Report Creator and the PLMXML Transformer. The implementer can register multiple reports with the same transformer or multiple transformer and vice-versa. The name of this plugin implementation will be used to populate a menu in Capital tools. Under the menus all the registered report names will be shown as separate action.

```
public interface IXDataExchangeRegistrar extends IXPlugin
{
    void register (IXDataExchangeRegistrarResult result);
}
```

register (IXDataExchangeRegistrarResult result): This method will be called to register all the Reports and their corresponding PLMXML transformer. For each entry added to the result, there will be corresponding action in Capital tool to generate the report and transform the data using the specified plugins.

@result: The result object to be populated by the implementer

Following code shows how to register the result:

```
result.add ("Export Harness ComponentBOM",
TeamcenterHarnessBOMToPLMXMLConverter.class,
TeamcenterHarnessComponentBOM.class);

TeamcenterHarnessBOMToPLMXMLConverter is the class implementing the
IXPLMXMLTransformer plugin.

TeamcenterHarnessComponentBOM is the class implementing the IXReportCreator
```

JSP tags

JSP tags give a declarative flavor to the interfaces for reporting. These tags wrap around the extensible interfaces for reporting and allows you to write JSP pages (reports especially) without introducing java code. The tag library reporter.tld of JSP tags is available for use by JSP pages in reporter.war(found in capital.home/reporter/servlet). A JSP referring to the tag library would need to add the following piece of code before using a jsp tag.

```
<%@ taglib prefix="chs" uri="/WEB-INF/reporter.tld" %>
```

Note that it also associates a prefix to the tags, so all JSP tags when used would need this prefix. (See example following the table).
JSP Tags for writing reports are :

Tag Name	Description
reportdefinition	This tag for defining a report is fully described by a title and a collection of tags report modules (a module corresponds to a single table). As a JSP page containing this tag runs, this tag constructs an IXReport object, evaluates it, obtaining scoping queries and report processor as session attributes and transforms the evaluated XML content to HTML using a default stylesheet(report.xsl). JSP Tags that it can contain are reportmoduledefinition. customattribute customtag The extensibility object constructed is IXReport
reportmoduledefinition	This tag for defining a report module (IXReportModule, which corresponds to a single table of a report) is described by • A title • A collection of queries for entities to show in this module, i.e. an IXQuery collection defined by queries tag. • A collection of report templates (IXReportTemplate) defined by reporttemplatedefinition tags. • Optionally it may have nested reportmodules, to handle cases where tables are nested inside other tables (see documentation for IXReportModule). • Optionally customattribute and custom tags can be provided to the module as hints to stylesheet that processes XML resulting from a report. JSP Tags that it can contain are therefore queries reportmoduledefinition. reporttemplatedefinition customatttibute customtag

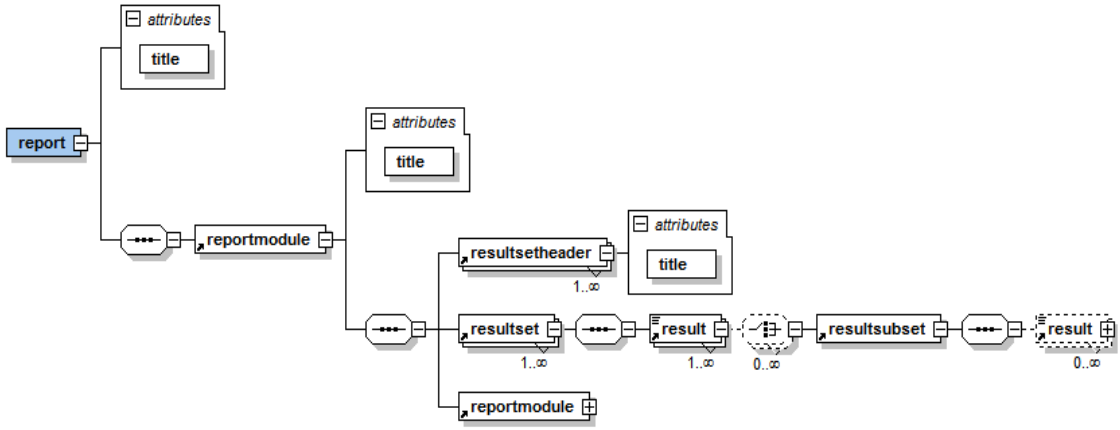
	After constructing an 'IXReportModule' object this tag appends to the list of modules in the parent report tag.
queries	This JSP tags a container for queries to be constructed in a report module. Therefore, this tag is fully described by the query tags it contains. This tag appends the list of queries to the parent reportmodule. The JSP tags that it can be nested within queries are : query
query	Query is defined by the entity type and class. It may also have an attribute "as" which is an alias you can use to distinguish results from other queries for the same entity. This tag can have entity tag within it and constructs an IXQuery object. A query tag can be provided a filter expression as raw xml nested within the tag or as java code that implements the IXFIlterExpression interface. The tags that can be nested within query tags are entity queryExpression(raw xml not a jsp tag)
entity	This tag is defined by a name and extensibility class corresponding to entity. Entity tag can appear under the query tag to define query.
reporttemplatedefinition	This tag is defined by a title and an expression (IXResultExpression) that will applies on each entity queried for by report module. An expression can be provided either as a queryexpression in the raw XML form or as JAVA code that implements the IXResultExpression interface. It constructs an IXReportTemplate object containing the result expression and adds it to the parent reportmoduledefinition tag.
customattribute	Custom attributes can be added to reporttemplatedefinition, reportmoduledefinition and reportdefinition and custom tags. They have a name and a value. These attributes are written to output xml inside the corresponding nodes. For example a custom attribute written to reportmoduledefinition will appear in the output xml as an attribute of reportmodule node. A customattribute on reporttemplatedefinition appear as an attribute on xml output node resultsetheader (column headers). These attributes can be processed by the style sheets (XSLT files).
customtag	Custom tags like custom attributes are useful information for style sheets that transform the output xml from reports. Custom tags can be added to reportmoduledefinition and reportdefinition. Custom tags are identified by a name and can have several attributes themselves, added using customattributes tag. On report evaluation, these tags

	with attributes appear in the output xml, with the node name the same as the name of the customtag.

Output XML Representation

In this section we discuss the schema for the report whose content is defined by the report definition we discussed so far. The schema will closely follow the definition as seen below

Report JSP tag Name	Generated Report Node Name(s)	Number	Description
reportdefinition	report	1	Root element
reportmoduledefinition	reportmodule	>=1	Content grouped by type of entity
reporttemplatedefinition	resultsetheader resultset/result or resultset/resultsubset/result	1 >=1	Report template (column) titles to resultsetheader nodes which form the headers for different columns. For each entity that results from queries in a module, we would find one resultset. The expression in a report template definition is used to evaluate content for each entity queried for in report module and placed in 'result'. A result could be a collection in which case the result node will contain a resultsubset.



Sample XML output:

```
<?xml version="1.0" encoding="UTF-8"?>
<report title="Harness report">
  <reportmodule title="">
    <groupBy title="Harness" showGroup="true"/>
    <reportdefinition title="Wire ID"/>
    <reportdefinition title="Length"/>
    <reportdefinition title="From"/>
    <reportdefinition title="To"/>
    <reportdefinition title="OptionExpression"/>
    <reportdefinition title="Multicore"/>
    <reportdefinition title="Harness" invisible="true"/>
    <reportdefinition title="Levels" spanAllCols="true"/>
    <resultset>
      <result>WIRE54</result>
      <result>1.75</result>
      <result> P4</result>
      <result>P10</result>
      <result/>
      <result>
        <resultsubset/>
      </result>
      <result>whole car_split4_split</result>
      <result>
        <resultsubset>
                <result>L1</result>
            <result>L2</result>
        </resultsubset>
      </result>
    </resultset>
  </reportmodule>
</report>
```

Custom tag

Column headers

Resultset corresponds to a Single row of a report with columns

Custom attributes on column

Column content can be just a string or a collection of strings.

Deployment of a report JSP:

After compilation JSP gets converted to .class files. For example wirelistdefinition.jsp gets compiled to wirelistdefinition_jsp.class in the build step class files are copied to WEB-INF\classes\resources\reports.

We still want to refer to these JSP files by their original names (for example as /reports/wirelistdefinition.jsp rather than /reports/wirelistdefintion_jsp) in the URL, so we need add entries in reporter\WEB-INF\web.xml to map servlets to URL name.

There are two entries you need to add, one that associates a name to the compiled JSP

```
<servlet>
     <servlet-name>wirelistdefinition_jsp</servlet-name>
     <servlet-class>resources.reports.wirelistdefinition_jsp
   </servlet-class>
</servlet>
```

and one that associates the name to a URL.

```
<servlet-mapping>
        <servlet-name>wirelistdefinition_jsp</servlet-name>
        <url-pattern>/reports/wirelistdefinition.jsp</url-pattern>
</servlet-mapping>
```

Next add a new jsp reference in the reports.XML file to make it available for UI to show them on screen.

```
<report jspFile='wirelistdefinition.jsp' name='Wire List'>
         <appliesOn designType='Logic Design'/>
         <appliesOn designType='Harness Design'/>
</report>
```

Server needs a restart after the deployment of the jsp.

Guidance

Here are some recommendations that will help you …

- For an example of a report processor that customizes the behavior of Connectivity between pinlists processor, see CustomReportProcessor.java in examples folder Capital installation.

- In reporter.war (in ${capital.home}/reporter/servlet) the file named sources.zip contains source code that you can use to customize reports.

- If you want to access a factory or a singleton instance, it is likely to be a session attribute or in the servlet context.

- The list of entity names and their extensibility classes that can be used to create queries are available in the interface IEntityNames.java found in the sources zip file .

- For creating inline filters for queries, you can use abstract class chs.web.reporter.utils.AnonymouseFilterExpression that implements all methods of IXFilterExpression apart from isSatisfiedBy.

- For creating inline result expressions that don't depend on a execution context, you can use abstract class chs.web.reporter.utils.ResultExpression that implements all methods of IXResultExpression apart from Object evaluate(IXObject entity).

- To prevent too many queries from hitting the server, you can keep a cache of values that you can refresh at certain intervals.

- Web applications should not keep extensibility objects across requests. They may be invalidated after the lifetime of a request.

- Web.xml has customizable parameters

    - Diagram image format could be switched between svg and png, using context-param 'imageformat'. Default value is svg.
    - Time interval between refresh of designs, using the context-param 'refreshIntervals'. Default value of refreshIntervals is 500s
    - Idle time after which http session is destroyed(session-timeout). Default value of session-timeout is 30min

Examples

Here is an example that prints the properties of all connectors of a logic design. It assumes that a designName and projectName are defined as session attributes

```
IXQueryFactory qf = (IXQueryFactory )session.getAttribute("queryFactory");
String projectName = (String)session.getAttribute("projectName");
String designName = (String) session.getAttribute("designName");
```

```
//create expression for selecting a project with a specific name
// AnonymousFilterExpression is a utility class available in chs.web.reporter.utils package in found in sources.zip. It provides default implementation for little used methods of IXFilterExpression(getName() and getCollapedForm()) that are not important for filters in a query.

IXFilterExpression projectFilter = new AnonymousFilterExpression()
        {
                public boolean isSatisfiedBy(IXObject entity)
                {
                return  entity.getAttribute("Name").equals(projectName);
                }
        });
IXFilterExpression designFilter = new AnonymousFilterExpression()
        {
                public boolean isSatisfiedBy(IXObject entity)
                {
                return  entity.getAttribute("Name").equals(designName);
                }
        });
```

```
//create a project query with a filter that picks a project with a specific name.
IXQuery projectQuery  = qf.createQuery( IXProject.class, "Project", projectFilter );
//create a design query with the filter
IXQuery designQuery = qf.createQuery(IXLogicDesign.class,"Logic Design", designFilter );

// appends the projectQuery as a related query to design  query, so that we look for designs only in the projects returned by the projectQuery.
designQuery.addRelatedQuery(projectQuery);


//create connector query with no filters( null)
IXQuery connectorQuery = qf.createQuery(IXConnector.class,"Connector",null);
//scope the connector query to those in the design.
```

```
connectorQuery.addRelatedQuery(designQuery);

// execute the query to fetch connectors in a design.
Collection<IXObject> result = connectorQuery.getResult();

// now get the list of properties from the connector
Set<String> propertyNames = new TreeSet<String>();
for (IXObject xobject : result) {
        Set<IXValue> properties = xobject.getProperties();
        for (IXValue value : properties) {
                propertyNames.add(value.getName());
        }
  }
}
```

Here is a sample wire list report for integrator designs with 3 columns. The first of which displays its name the second and third both display levels by two different methods

```
<chs:reportdefinition title="Wire List">

<chs:reportmoduledefinition title="">

    <chs:queries>
      <chs:query>
          <chs:entity name="Wire" class ="com.mentor.chs.api.IXWire">
         </chs:entity>
      </chs:query>
    </chs:queries>
<!—column 1 displays the name attribute-->
    <chs:reporttemplatedefinition title="Name" >
     <queryExpression>
            <attributeVariable name="Name"/>
     </queryExpression>
    </chs:reporttemplatedefinition >

<!—column 2 displays the name of all the levels -->
    <chs:reporttemplatedefinition title="Levels">
        <queryExpression>
            <apply>
              <related>
                  <stringConstant value="Harness Level">
               </related>
              <attributeVariable name="Name"/>
            </apply>
        </queryExpression>
     </chs:reporttemplatedefinition>

<!—column 3 displays the name of the levels using an ResultExpression that is created
inline -->
```

Define types of entities to show in a report module, within query tags.

A reporttemplatedefinition tag defines columns of a table by an expression evaluated on each of wire which could be xml representation of query expression or JAVA code. As shown in the next reporttemplatedefinition tag

```
<chs:reporttemplatedefinition title="Levels" expression="<%=
new ResultExpression()
    {
        public Object evaluate(IXObject entity)
        {
            Collection<String> harnessLevelNames = new ArrayList<String>();
            if( entity instanceof IXWire) {
                Set<IXHarnessLevel> levels = ((IXWire) entity).getHarnessLevels();
                for (IXHarnessLevel level : levels) {
                    harnessLevelNames.add( level.getName());
                }
            }
            return harnessLevelNames;
        }

    };
%>" >

</chs:reporttemplatedefinition></chs:reportmoduledefinition>
</chs:reportdefinition>
```

Grouping and Sorting using Custom Tags:

Grouping is typically used in BOM reporting. Report module content can grouped based on certain columns or some expressions. There could also be multiple columns /expressions according to which to report module could be grouped

In this example we show Harness Component BOM which by Part Number of each entity reported. To handle this we add a custom tag 'groupBy' (as many as needed) under reportmoduledefinition as below:

Whether content in a group should be expanded and shown in the UI or whether representative elements should only be seen is handled by custom attributes. In this case, we want to expand components grouped by Design to show all entities under design but collapse content to just one representative element to while grouping by Part Number. This is typically what happens in Component BOMs where grouped data is collapsed to one representative element whose Part Number is displayed.

```
<chs:reportdefinition title="Harness Component BOM">


<chs:reportmoduledefinition title=" ">
    <queries>
            <!-- queries go here-->
    </queries>


    <chs:customtag name="groupBy">
        <chs:customattribute name="title" value="PartNumber"/>
```

Custom tag groupBy has attributes title and expandGroup

```
    </chs:customtag>


    <chs:reporttemplatedefinition title="Part Number">
            <queryExpression>
                <attributeVariable name="PartNumber" />
            </queryExpression>
     </chs:reporttemplatedefinition>

     <chs:reporttemplatedefinition title="Quantity">
           <queryExpression>
                 <integerConstant value="1"/>
          </queryExpression>
    </chs:reporttemplatedefinition>

</chs:reportmoduledefinition>
</chs:reportdefinition>
```

Sorting of grouped data is done by adding sort tag(s) as attributes to groupBy element

For reports without grouping, sorting can be done with respect to an existing report template title as the following example shows.
(where WireList is sorted according to Wire ID)

```
<chs:reportdefinition>
  <chs:reportmoduledefinition title = "WireList">

     <chs:customtag name = "sortBy" >
        < chs:customattribute name = "title"  value = "Wire ID">
         </chs:customattribute>
    </chs:customtag>

    < chs:queries>
      <chs:query>
          <chs:entity name="Wire" class="com.mentor.chs.api.IXWire">
         </chs:entity>
      </chs:query>
    </chs:queries>

  <chs:reporttemplatedefinition title="Wire ID">
         <queryExpression id="" name="WireName"  modified="0" baseid="">
             <attributeVariable name="Name"/>
         </chs:queryExpression>
      </chs:reporttemplatedefinition>
     <!--  rest of the module  content -->
   </chs:reportmoduledefinition>
</chs:reportdefinition>
```

Custom Attribute

We already encountered custom attribute inside tags while grouping. Custom Attributes help are not as useful to report generator, as to a postprocessor (style sheet) that works on the generated report xml-content.

Some of the tags useful for generating reports are processed by the default style sheet we provide are:

Tag	Values	Description
accumulateUsing	(+)	Used to sum values of grouped (or otherwise) data of a report template, to show total values.
expandGroup	(true|false)	if true, data within group is shown, else representative element only
spanAllCols	(true | false)	This is useful to force column content to assume space in the next row in a table.
Float	(true|false)	This is useful for the formatter if it wants recognize float values so that it can round off to certain decimal places

An example below shows the use of the tag: spanAllCols:

Harness Levels could be long list to show in a single column of a wire list report, so we tag the report template with spanAllCols tag.

```
<chs:reportmoduledefinition title="Wire List">
      <chs:queries>
                <!-- queries go here-->
       </chs:queries>

         <!-- other column report data go here like Wire ID etc -->

         <chs:reporttemplatedefinition title="Levels" >

         <chs:customattribute name="spanAllCols" value ="true" >
         </chs:customattribute>

           <queryExpression name="Levels" >
              <apply>
```

Levels is a report template with the attribute spanAllCols set to true indicating to stylesheet that this element should be shown in a row of its own for each wire in the wire list spanning all the columns

```
                <related>
                <stringConstant value="Harness Level"/>
                </related>
                <attributeVariable name="Name"/>
              </apply>
            <queryExpression>
          </chs:reporttemplatedefinition>
</chs:reportmoduledefinition>
```

Report	Wire list	
User	*Not Available*	

Wire ID	Length	From	To	OptionExpression	Multicore
WIRE3	2.5		P1		
Levels					
_m1,_m2,_m3					
WIRE1	5.25		P2		
Levels					
_m1,_m2					
WIRE4	6.25		P6		
Levels					
_m1					
WIRE2	6.25		P3		
Levels					
_m1					

Here is an example that uses the accumulateUsing tag in component BOM:

Here Quantity in a component BOM is set to 1 for each entity type to report on and the letting style sheet handle tag accumulateUsing to appropriately sum values for each group of entities corresponding to a single Part Number.

```
< chs:reportmoduledefinition title = "Component BOM">

        <chs:customtag name = "groupBy" value="Part Number">
       </chs:customtag>

       <chs:queries>
         <chs:query>
           <chs:entity name = "Connector" class="com.mentor.chs.api.IXConnector">
          </chs:entity>
         </chs:query>
```

```
        <chs:query>
            <entity name = "Splice" class="com.mentor.chs.api.IXSplice"/>
        </chs:query>
    <!—rest of the queries go here-->
     </chs:queries>
      <!—Column 1-->
    <chs:reporttemplatedefinition title="Part Number">
        <queryExpression >
            <attributeVariable name="PartNumber"/>
        </queryExpression>
    </chs:reporttemplatedefinition>

      <!—Column 2-->
     <chs:reporttemplatedefinition title="Quantity>
        <chs:customattribute name= “accumulateUsing” value="+">
        </chs:customattribute>
         <queryExpression >
            <integerConstant value="1"/>
         </queryExpression>
    </chs:reporttemplatedefinition>
</chs:reportmoduledefinition>
```

Report	Component BOM
User	*Not Available*

Part Number	Quantity
C-64451	1
C-001 new	1
ModularConn5	1
C-005	2
C-60713	1
10 way_new	1
SW-51167	2
2594-Wire-O	1
SW-51125	1
2594-Wire-G	1
SW-51095	1
SW-51173	1
SW-51067	1
2594-Wire-K	1
SW-51003	1
SW-51089	2
SW-51092	1

Quantity is a report template with the tag: accumulateUsing set to ‘+’ indicating to the styleshet to sum content for each group.

W-0004	1
SW-51054	1

The ‘expandGroup’ attribute in ‘groupBy’ tag to show content inside group:

```
<chs:reportdefinition title="Harness Report" >
  <chs:reportmoduledefinition >
    <chs:customtag name="groupBy">

      <chs:customattribute name="title" value="Design">
      </chs:customattribute>

      <chs:customattribute name="expandGroup" value="true"/>
      </chs:customattribute>
    </chs:customtag>
```

When true, content is expanded to show all data within.

Report	Harness report
User	*Not Available*

Harness : H1

Harness	#Connectors	#Inlines	#Splices	#Wires	Length
H1	0	0	0	1	6.250
H1	0	0	0	1	6.250
H1	0	0	0	1	5.250
H1	1	0	0	0	0
H1	1	0	0	0	0
H1	1	0	0	0	0

Harness : H6

Harness	#Connectors	#Inlines	#Splices	#Wires	Length
H6	0	0	0	1	2.50
H6	1	0	0	0	0
H6	1	0	0	0	0

When expandGroup is false content is collapsed: Note also that Lengths are accumulated.

Report	Harness report
User	*Not Available*

Harness	#Connectors	#Inlines	#Splices	#Wires	Length
H6	2	0	0	1	2.5
H1	3	0	0	3	17.75

Custom Embedded Report Plugin Development

At this point, you should have a good idea how to implement this type of plugin. Following is the examples to write Wire List Report with column and color that applies only for logic designs

```
/**
 * The application will use this plugin to create a report using the method createReport.
 *  The report is then evaluated to produce xml string. It then transforms the xml string using the
 * specified xslt source.
 *
 */
public class WireReport  implements IXLogicReport
{

	public StreamSource getStyleSheet()
	{
		String chs_home = System.getenv("CAPITAL_HOME");
		String styleSheetFileName = chs_home.replace("\\", "/") + File.separator +
"/reporter/templates/report.xsl";
		File file = new File(styleSheetFileName);
		return new StreamSource(file);
	}

	public IXReport createReport(IXReportContext context)
	{
		IXReport report = null;
		try {
			// wireQuery for design of which we wish to see the wire list report

			IXReportFactory rf = context.getReportFactory();
			IXQueryFactory qf = context.getQueryFactory();
			IXQuery wireQuery = qf.createQuery(IXWire.classS, “Wire”, (IXFilterExpression) null);

			IXReportModule module = context.getReportFactory().createReportModule("");

			module.addQuery(wireQuery, "");

			report = rf.createReport("Wire List Report");
			report.addModule(module);
			IXReportTemplate column = rf.createReportTemplate( new IXResultExpression(){
				public String getName()
				{
					return "";
				}

				public Object evaluate(IXObject entity)
				{
```

```
                                        return entity.getAttribute("Name");
                                }

                                public Object evaluate(IXObject entity, String executionContext)
                                {
                                        return evaluate(entity);
                                }}, "Name");
                        module.addReportTemplate(column);


                        IXReportTemplate column = rf.createReportTemplate( new IXResultExpression(){
                                public String getName()
                                {
                                        return "";
                                }

                                public Object evaluate(IXObject entity)
                                {
                                        return entity.getAttribute("Color");
                                }

                                public Object evaluate(IXObject entity, String executionContext)
                                {
                                        return evaluate(entity);
                                }}, Color);
                        module.addReportTemplate(column);

                }
                catch (Exception e) {
                        e.printStackTrace();
                }

                return report;
        }



        public String getDescription()
        {
                return “Wire List”;
        }

        public String getName()
        {
                return “Wire List Report”;
        }

        public String getVersion()
        {
                return “WireList”;
        }
}
```

The examples provided may be different in future releases of Capital. If you do use these examples, therefore, you need make a copy of them (we reserve the right to add, remove or modify these examples).

Appendix A

List of extensibilities classes that can be accessed through queries:

com.mentor.chs.api.IXBackshell,
com.mentor.chs.api.IXBackshellTermination
com.mentor.chs.api.IXBundle
com.mentor.chs.api.IXBundleRegion
com.mentor.chs.api.IXCavity
com.mentor.chs.api.IXDeviceConnector
com.mentor.chs.api.IXConnector
com.mentor.chs.api.IXInterconnectDevice
com.mentor.chs.api.IXGround
com.mentor.chs.api.IXDevice
com.mentor.chs.api.IXDevicePin
com.mentor.chs.api.IXDevicePin
com.mentor.chs.api.IXGround
com.mentor.chs.api.IXHarness
com.mentor.chs.api.IXHarnessLevel
com.mentor.chs.api.IXHole
com.mentor.chs.api.IXInterconnect
com.mentor.chs.api.IXMulticore
com.mentor.chs.api.IXNet
com.mentor.chs.api.IXShield
com.mentor.chs.api.IXSignal
com.mentor.chs.api.IXSlot
com.mentor.chs.api.IXSplice

com.mentor.chs.api.IXSplicePin
com.mentor.chs.api.IXWire
com.mentor.chs.api.IXCavityDetail
com.mentor.chs.api.IXWireEnd
com.mentor.chs.api.IXInsulationRun
com.mentor.chs.api.IXInsulation
com.mentor.chs.api.IXSpotTape
com.mentor.chs.api.IXBreakoutTape
com.mentor.chs.api.IXMultiLocationComponent
com.mentor.chs.api.IXGrommet
com.mentor.chs.api.IXClip
com.mentor.chs.api.IXOtherComponent
com.mentor.chs.api.IXReferenceNode
com.mentor.chs.api.IXStructureNode
com.mentor.chs.api.IXHarnessRegister
com.mentor.chs.api.IXAdditionalComponent
com.mentor.chs.api.IXLibraryConnector
com.mentor.chs.api.IXLibraryObject
com.mentor.chs.api.IXLibraryComponentTypeCode
com.mentor.chs.api.IXLibraryColorCode
com.mentor.chs.api.IXLibraryMaterialCode
com.mentor.chs.api.IXAssembly
com.mentor.chs.api.IXOption
com.mentor.chs.api.IXProject
com.mentor.chs.api.IXBuildList
com.mentor.chs.api.IXDiagram
com.mentor.chs.api.IXLogicDesign
com.mentor.chs.api.IXIntegratorDesign
com.mentor.chs.api.IXHarnessDesign
com.mentor.chs.api.IXMarker